

# Transactions in Entity Framework Core (EF Core)

In this article, I will discuss **Transactions in Entity Framework Core (EF Core)** with Examples. Please read our previous article discussing [Entity Framework Core Inheritance \(TPH, TPT, and TPC\)](#) with Examples.

## What are Transactions in Entity Framework Core?

Entity Framework Core (EF Core) is a modern, open-source, cross-platform Object-Relational Mapper (ORM) for .NET. It allows developers to work with a database using .NET objects, eliminating much of the data-access code that we usually need to write.

Transactions in EF Core ensure that a group of database operations (multiple Data Manipulation Language – DML operations) is treated as a single unit of work. This means that either all operations succeed or none of them are applied, maintaining the integrity and consistency of data in the database. This also means that if one operation in a sequence fails, all previous operations within the same transaction are rolled back. This means the job is never half done; either all of it is done, or nothing is done.

## Different Ways to Implement Transactions in Entity Framework Core

EF Core provides several mechanisms to implement transactions, allowing developers to choose the most suitable approach based on the specific requirements of their application. The primary methods include:

- **Implicit or Automatic Transactions with SaveChanges():** By default, EF Core wraps operations in SaveChanges() within a transaction. If any command fails, it throws an exception, and all changes are rolled back.
- **Manual Transactions with Database.BeginTransaction():** Manually control transaction boundaries for multiple operations. That means we can manually begin, commit, or rollback transactions using the Database property of the DbContext instance.
- **Asynchronous Transactions:** We can also manage transactions asynchronously using asynchronous methods, which is useful in web applications where we want to manage transactions without blocking threads.
- **Distributed Transactions with TransactionScope:** Automatically manage transactions that span multiple contexts or databases.

Let's proceed and understand each method in detail with real-time examples.

## Real-World Example to Understand Transactions with Entity Framework Core:

Consider an e-commerce application where a customer places an order consisting of multiple items. Placing an order involves multiple operations, such as creating an order record, updating inventory, and processing payment. It's important to note that all the operations are saved successfully; otherwise, the entire operations should be rolled back to maintain data integrity and consistency.

Let us proceed and implement this example using Transactions in Entity Framework Core with all the approaches. First, let us create the Entities required for our application.

### Order Entity:

Create a class file named **Order.cs** within the Entities folder and then copy and paste the following code. The following entity represents a customer's order in the e-commerce application.

```
using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
```

```

public class Order
{
    public int OrderId { get; set; } // Primary Key
    public DateTime OrderDate { get; set; }
    [Column(TypeName = "decimal(18,2)")]
    public decimal TotalAmount { get; set; }
    // Navigation Property - One Order has many OrderItems
    public ICollection<OrderItem> OrderItems { get; set; }
    // Navigation Property - One Order has one Payment
    public Payment Payment { get; set; }
}
}

```

### OrderItem Entity:

Create a class file named **OrderItem.cs** within the Entities folder, and then copy and paste the following code. The following entity represents an individual item within an order.

```

using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    public class OrderItem
    {
        public int OrderItemId { get; set; } // Primary Key
        public int ProductId { get; set; } // Foreign Key to Product
        public int Quantity { get; set; }
        [Column(TypeName = "decimal(18,2)")]
        public decimal Price { get; set; }
        public int OrderId { get; set; } // Foreign Key to Order
        // Navigation Properties
        public Order Order { get; set; }
        public Product Product { get; set; }
    }
}

```

### Product Entity:

Create a class file named **Product.cs** within the Entities folder and copy and paste the following code. The following entity represents the product inventory in the e-commerce system.

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    public class Product
    {
        public int ProductId { get; set; } // Primary Key
        [MaxLength(100)]
        public string Name { get; set; }
        [Range(0, double.MaxValue)]
    }
}

```

```

        [Column(TypeName = "decimal(18,2)")]
        public decimal Price { get; set; }
        [Range(0, int.MaxValue)]
        public int Quantity { get; set; }
        public string? Description { get; set; }
        public bool IsInStock { get; set; }
        // Navigation Property - One Product has many OrderItems
        public ICollection<OrderItem> OrderItems { get; set; }
        // Concurrency Token
        [Timestamp]
        public byte[] RowVersion { get; set; }
    }
}

```

## Payment Entity:

Create a class file named **Payment.cs** within the Entities folder and then copy and paste the following code. The following entity represents a payment transaction for an order.

```

using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    public class Payment
    {
        public int PaymentId { get; set; } // Primary Key
        public int OrderId { get; set; } // Foreign Key
        public DateTime PaymentDate { get; set; }
        [Column(TypeName = "decimal(18,2)")]
        public decimal Amount { get; set; }
        public PaymentStatus Status { get; set; } // Pending, Failed,
Completed
        // Navigation Property - One Payment belongs to one Order
        public Order Order { get; set; }
    }
    public enum PaymentStatus
    {
        Pending,
        Failed,
        Completed
    }
}

```

## OrderLog Entity:

Create a class file named **OrderLog.cs** within the Entities folder, and then copy and paste the following code. The following entity represents a log entry for order-related activities, typically stored in a separate logging database.

```

namespace EFCoreCodeFirstDemo.Entities
{
    public class OrderLog

```

```

    {
        public int OrderLogId { get; set; } // Primary Key
        public int OrderId { get; set; } // Foreign Key
        public DateTime LogDate { get; set; }
        public string Message { get; set; }
    }
}

```

## DbContext Classes

To manage the entities and their interactions with the database, two separate DbContext classes are used in our application.

### ECommerceDbContext

Manages the primary e-commerce entities such as Orders, Order Items, Products, and Payments. So, create a class file named ECommerceDbContext.cs within the Entities folder and copy and paste the following code.

```

using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class ECommerceDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // Configure the connection string to your ECommerceDB SQL Server
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=ECommerceDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        // Configures the model and mappings between entities and database
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // Configure enums to be stored as strings
            // For PaymentStatus enum
            modelBuilder.Entity<Payment>()
                .Property(c => c.Status)
                .HasConversion<string>()
                .IsRequired(); // Optional: Specify if the property is required
        }
        // DbSet for primary e-commerce entities
        public DbSet<Order> Orders { get; set; }
        public DbSet<OrderItem> OrderItems { get; set; }
        public DbSet<Product> Products { get; set; }
        public DbSet<Payment> Payments { get; set; }
    }
}

```

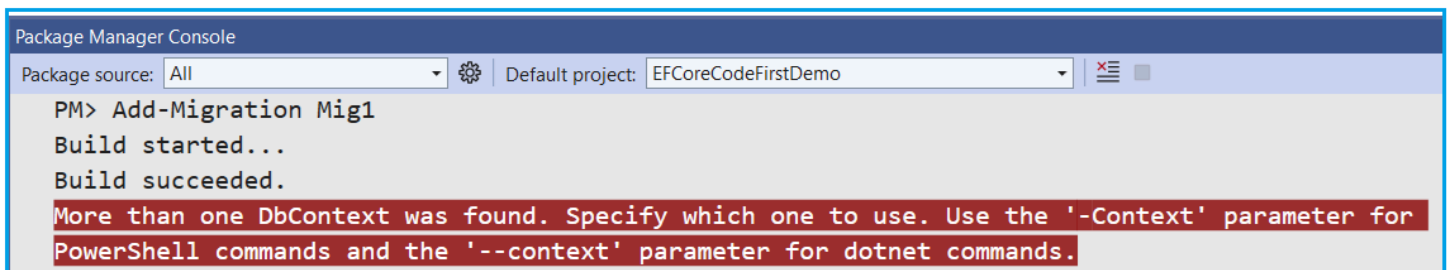
### LoggingDbContext

This context class manages logging entities such as OrderLog and typically connects to a separate logging database. So, create a class file named LoggingDbContext.cs within the Entities folder and then copy and paste the following code.

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class LoggingDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // Configure the connection string to your Logging SQL Server Database
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-6P5NK25R\SQLSERVER2022DEV;Database=LoggingDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        // DbSet for logging entities
        public DbSet<OrderLog> OrderLogs { get; set; }
    }
}
```

## Generate Migration and Update the Databases:

Open the Package Manager Console and execute the **Add-Migration** and **Update-Database** commands as follows, but you should get one error.



EF Core uses DbContext classes to interact with the database. When multiple DbContext classes are present in a project, EF Core commands (like Add-Migration or Update-Database) need to know which context to target. Without specifying, EF Core cannot determine the correct context, leading to the error: **More than one DbContext was found. Specify which one to use. Use the '-Context' parameter for PowerShell commands and the '--context' parameter for dotnet commands.**

## How to Solve the above Error?

To solve the above error, we need to specify the DbContext to target using parameters in the commands. For example, when using the Package Manager Console in Visual Studio, you can specify the context with the -Context parameter. The syntax is given below:

- **Add-Migration MigrationName -Context YourDbContextName**
- **Update-Database -Context YourDbContextName**

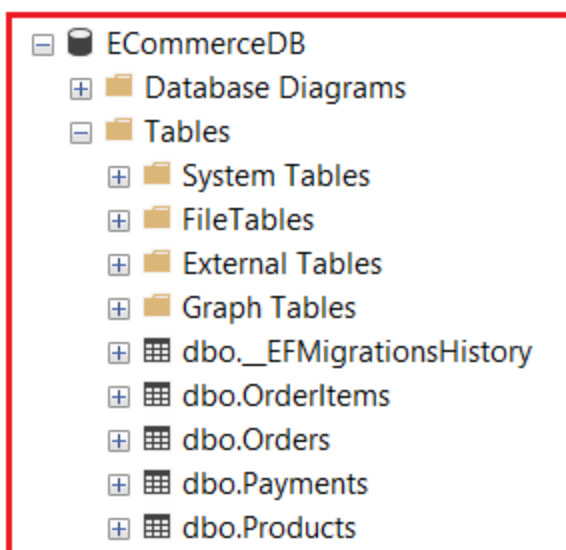
So, let us proceed with Generating and Applying the migration for both of our DbContext classes. For ECommerceDbContext, please use **Add-Migration InitialCreate -Context ECommerceDbContext** and **Update-Database -Context ECommerceDbContext** commands in the Package Manager Console as shown in the below image:

```
Package Manager Console
Package source: All [v] [g] Default project: EFCoreCodeFirstDemo [v]
PM> Add-Migration InitialCreate -Context ECommerceDbContext
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> Update-Database -Context ECommerceDbContext
Build started...
Build succeeded.
Applying migration '20240925095539_InitialCreate'.
Done.
```

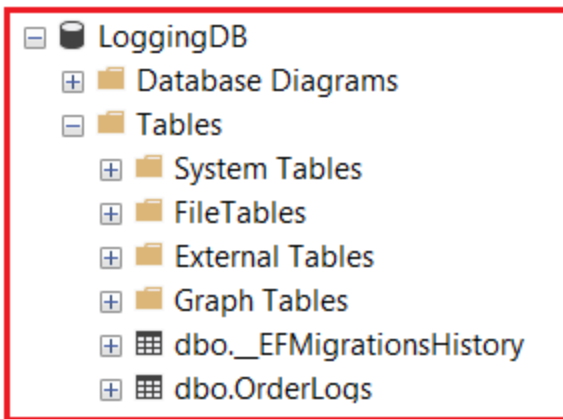
For LoggingDbContext, please use **Add-Migration InitialLogging -Context LoggingDbContext** and **Update-Database -Context LoggingDbContext** commands in the Package Manager Console as shown in the below image:

```
Package Manager Console
Package source: All [v] [g] Default project: EFCoreCodeFirstDemo [v]
PM> Add-Migration InitialLogging -Context LoggingDbContext
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> Update-Database -Context LoggingDbContext
Build started...
Build succeeded.
Applying migration '20240925095650_InitialLogging'.
Done.
```

Now, if you verify the database, then we should have the ECommerceDB database with the following tables:



Similarly, we should have the LoggingDB database created with the following tables:



## Insert Statements for the Products Table of the ECommerceDB database

To facilitate testing and demonstrate the functionality, please execute the following SQL INSERT statements to populate the Products table in the ECommerceDB database.

```
-- Insert dummy products into the Products table
INSERT INTO Products (Name, Price, Quantity, Description, IsInStock)
VALUES
('Laptop', 1500.00, 10, 'High-performance laptop suitable for all purposes.', 1),
('Smartphone', 800.00, 20, 'Latest model smartphone with advanced features.', 1),
('Tablet', 600.00, 15, 'Lightweight tablet with high-resolution display.', 1),
('Headphones', 200.00, 50, 'Noise-cancelling over-ear headphones.', 1),
('Monitor', 300.00, 25, '24-inch full HD monitor with vibrant colors.', 1);
```

## Implicit Transactions with SaveChanges() in EF Core

By default, EF Core wraps all changes made in `SaveChanges()` within a transaction. If any operation within `SaveChanges()` fails, EF Core automatically rolls back the entire transaction, ensuring no partial changes persist. Let us place an Order Using EF Core Implicit Transaction. So, modify the Program class as follows:

```
using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            try
            {
                // Initialize the DbContext for interacting with the
                ECommerce database
                using var context = new ECommerceDbContext();
                Console.WriteLine("Creating an Order");
            }
        }
    }
}
```

```

        // 1. Create a new order with multiple order items and an
initial payment
        var order = new Order
        {
            // Set the order date to the current UTC time
            OrderDate = DateTime.UtcNow,
            // Set the total amount for the order
            TotalAmount = 250.00m,
            // Define the order items associated with this order
            OrderItems = new List<OrderItem>
            {
                // First item in the order
                new OrderItem
                {
                    ProductId = 1, // Refers to the product with ID 1
                    Quantity = 2,  // 2 units ordered
                    Price = 100.00m // Price per unit
                },
                // Second item in the order
                new OrderItem
                {
                    ProductId = 2, // Refers to the product with ID 2
                    Quantity = 1,  // 1 unit ordered
                    Price = 50.00m // Price per unit
                }
            },
            // Define the payment details associated with the order
            Payment = new Payment
            {
                PaymentDate = DateTime.UtcNow, // Set the payment
date as the current UTC time
                Amount = 250.00m, // Payment amount matches the total
order amount
                Status = PaymentStatus.Pending // Set the initial
payment status to 'Pending'
            }
        };
        // Add the new order to the DbSet in the context (but not yet
saved to the database)
        context.Orders.Add(order);
        Console.WriteLine("Order added to the DbContext... Preparing
to save changes.");
        // 2. Save the changes to the database, which inserts the
order, order items, and payment
        context.SaveChanges();
        Console.WriteLine($"Order placed successfully with Order ID:
{order.OrderId}.");
    }
    catch (DbUpdateException ex)
    {

```

```

        // Catch database update errors, such as constraint
violations or connection issues
        Console.WriteLine($"Database Error placing order:
{ex.InnerException?.Message ?? ex.Message}");
        // Any error during SaveChanges() automatically rolls back
the transaction
    }
    catch (Exception ex)
    {
        // Catch any general exceptions that occur during the process
        Console.WriteLine($"Error placing order: {ex.Message}");
        // If an exception occurs, the changes are rolled back, and
the database state remains consistent
    }
}
}
}

```

### Explanation:

When `SaveChanges()` is called, EF Core starts a transaction. It inserts the `Order` and associated `OrderItem` records. EF Core automatically rolls back the transaction if any operation fails (e.g., a constraint violation), ensuring that no partial data is saved. This approach is simple, straightforward, and suitable for simple scenarios where all operations are within a single `SaveChanges()` call. When you run the above code, you will get the following output:

```

Creating an Order
Order added to the DbContext... Preparing to save changes.
Order placed successfully with Order ID: 1.

```

### Manual Transactions using `Database.BeginTransaction()` in EF Core

For scenarios requiring more control over the transaction boundaries, such as multiple `SaveChanges()` calls within a single transaction, EF Core allows manual transaction management using the `Database.BeginTransaction()` method.

Let us understand this with an example. Let us place an `Order`, with `Product Quantity Update` and `Payment Processing`. So, we will do the following:

- Creating an order and order items.
- Updating `Product Quantity` for the ordered products.
- Processing the payment.

Each of these operations involves separate `SaveChanges()` calls but needs to be part of a single transaction. For a better understanding, please modify the `Program` class as follows:

```

using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {

```

```

static async Task Main(string[] args)
{
    // Initialize the DbContext for interacting with the ECommerce
    database
    using var context = new ECommerceDbContext();
    // Begin a manual transaction using the context
    using var transaction = await
context.Database.BeginTransactionAsync();
    Console.WriteLine("Transaction started...");
    try
    {
        // 1. Create a new order with order items and initial payment
        status (Pending)
        var order = new Order
        {
            OrderDate = DateTime.UtcNow, // Set the current UTC time
            for the order date
            TotalAmount = 150.00m, // Set the total amount of the
            order
            // Define the items in the order (in this case, one item)
            OrderItems = new List<OrderItem>
            {
                new OrderItem
                {
                    ProductId = 1, // The ProductId of the item being
                    ordered
                    Quantity = 1, // Number of units ordered
                    Price = 150.00m // Price per unit
                }
            },
            // Define the payment details associated with the order
            Payment = new Payment
            {
                PaymentDate = DateTime.UtcNow, // Set the payment
                date as the current UTC time
                Amount = 150.00m, // Set the payment amount to match
                the order total
                Status = PaymentStatus.Pending // Set the initial
                payment status to 'Pending'
            }
        };
        // Add the new order to the Orders DbSet in the context
        context.Orders.Add(order);
        Console.WriteLine("Order and associated payment details added
to the DbContext...");
        // Save changes asynchronously to the database (this inserts
        the order, items, and payment)
        await context.SaveChangesAsync();
        Console.WriteLine($"Order ID {order.OrderId} created with
initial payment status '{order.Payment.Status}'.");
        // 2. Update the product inventory based on the ordered item

```

```

        var orderItem = order.OrderItems.First(); // Get the first
(and only) order item
        var product = await context.Products.FirstOrDefaultAsync(p =>
p.ProductId == orderItem.ProductId); // Fetch the product from the database
        if (product != null)
        {
            // Reduce the product's quantity based on the ordered
quantity
            product.Quantity -= orderItem.Quantity;
            // Update the 'In Stock' status based on the new quantity
            product.IsInStock = product.Quantity > 0;
            // Update the product in the context
            context.Products.Update(product);
            Console.WriteLine($"Updating inventory for product
'{product.Name}'...");
            // Save changes asynchronously to reflect the updated
inventory in the database
            await context.SaveChangesAsync();
            Console.WriteLine($"Product '{product.Name}' inventory
updated. New Quantity: {product.Quantity}. In Stock: {product.IsInStock}");
        }
        else
        {
            // If the product is not found, throw an exception to
trigger a rollback
            throw new Exception("Product not found in the
inventory.");
        }
        // 3. Update the payment status to 'Completed' after
successful inventory update
        order.Payment.Status = PaymentStatus.Completed; // Update the
payment status to 'Completed'
        context.Payments.Update(order.Payment); // Update the payment
entity in the context
        Console.WriteLine($"Updating payment status for Order ID
{order.OrderId}...");
        // Save changes asynchronously to update the payment status
in the database
        await context.SaveChangesAsync();
        Console.WriteLine($"Payment status for Order ID
{order.OrderId} updated to '{order.Payment.Status}'.");
        // Commit the transaction if all operations are successful
        await transaction.CommitAsync();
        Console.WriteLine($"Transaction committed successfully for
Order ID {order.OrderId}.");
    }
    catch (DbUpdateException ex)
    {
        // Rollback the transaction if a database update error occurs
        await transaction.RollbackAsync();
    }
}

```

```

        Console.WriteLine($"Transaction rolled back due to Database
Error: {ex.InnerException?.Message ?? ex.Message}");
    }
    catch (Exception ex)
    {
        // Rollback the transaction if any other general error occurs
        await transaction.RollbackAsync();
        Console.WriteLine($"Transaction rolled back due to an error:
{ex.Message}");
    }
}
}
}
}

```

## Explanation:

- **Begin Transaction:** The transaction is started using `BeginTransaction()`.
- **Multiple Operations:** Multiple `SaveChanges()` calls are made within the transaction.
  1. Create Order and OrderItem.
  2. Update Inventory.
  3. Process Payment.
- **Commit/Rollback:** If all operations succeed, `transaction.Commit()` is called to commit the transaction. If any operation fails, `transaction.Rollback()` is invoked to undo all changes.

This approach allows grouping multiple operations into a single transactional unit, ensuring that all operations succeed or fail. So, when you run the above code, you will get the following output:

```

Transaction started...
Order and associated payment details added to the DbContext...
Order ID 2 created with initial payment status 'Pending'.
Updating inventory for product 'Laptop'...
Product 'Laptop' inventory updated. New Quantity: 9. In Stock: True
Updating payment status for Order ID 2...
Payment status for Order ID 2 updated to 'Completed'.
Transaction committed successfully for Order ID 2.

```

## Distributed Transactions with TransactionScope in Entity Framework Core

When operations span multiple `DbContext` instances or different databases, `TransactionScope` can manage distributed transactions. This ensures that all participating operations across different contexts or resources are committed or rolled back together. Distributed transactions involve more overhead and should be used only when necessary.

Let us understand this with an example. When placing an order, we also need to log the activity in a separate logging database. Both operations need to be part of a single transaction. For a better understanding, please modify the `Program` class as follows:

```

using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
using System.Transactions;
namespace EFCoreCodeFirstDemo
{

```

[illegible]

```

    }
    },
    // Create a payment entry for the order
    Payment = new Payment
    {
        PaymentDate = DateTime.UtcNow, // Set the
payment date to the current UTC time
        Amount = 300.00m, // Payment amount matches
the total order amount
        Status = PaymentStatus.Pending // Initial
status of the payment is 'Pending'
    }
};
// Add the newly created order to the Orders DbSet
orderContext.Orders.Add(order);
Console.WriteLine("Order and associated details added
to the DbContext...");
// Save changes to the database to insert the order,
order items, and payment into the database
orderContext.SaveChanges();
Console.WriteLine("Order saved to the database.");
// Store the generated OrderId for logging purposes
generatedOrderId = order.OrderId;
Console.WriteLine($"Order ID {order.OrderId} created
with initial payment status '{order.Payment.Status}'");
}
// 2. Perform logging operations using LoggingDbContext
using (var logContext = new LoggingDbContext())
{
    // Create a new OrderLog entry to record the
successful placement of the order
    var log = new OrderLog
    {
        OrderId = generatedOrderId, // Use the generated
OrderId for the log entry
        LogDate = DateTime.UtcNow, // Set the log date to
the current UTC time
        Message = "Order placed successfully." //
Descriptive log message
    };
    // Add the log entry to the OrderLogs DbSet
logContext.OrderLogs.Add(log);
Console.WriteLine("Log entry for the order
created...");
    // Save the log entry to the logging database
logContext.SaveChanges();
Console.WriteLine($"Log entry saved for Order ID
{log.OrderId}.");
}
// 3. Mark the transaction as complete
scope.Complete();

```

```

        Console.WriteLine("Transaction successfully completed.
Order placed and logged.");
    }
    catch (DbUpdateException ex)
    {
        // Handle database-related exceptions such as constraint
        violations or connection issues
        Console.WriteLine($"Database error occurred:
{ex.InnerException?.Message ?? ex.Message}");
        // No explicit rollback is needed here. Disposing the
        TransactionScope without calling Complete() will automatically roll back the
        transaction.
    }
    catch (Exception ex)
    {
        // Handle any other general exceptions that may occur
        Console.WriteLine($"An error occurred: {ex.Message}");
        // The transaction will be rolled back automatically upon
        disposal if Complete() is not called.
    }
    // At the end of the using block, the TransactionScope is
    disposed. If Complete() was not called, the transaction is rolled back
    automatically.
}
}
}
}
}

```

### Explanation:

- **TransactionScope:** Encapsulates a block of code in a transaction.
- **Multiple DbContexts:** Operations are performed across two different DbContext instances (ECommerceDbContext and LoggingDbContext) connected to different databases.
- **Atomicity:** If any operation within the TransactionScope fails, all changes across both contexts are rolled back.
- **scope.Complete():** Must be called to commit the transaction. If not called, the transaction is rolled back upon disposal.

Now, run the above code, you should get the following output:

```

Order and associated details added to the DbContext...
Order saved to the database.
Order ID 3 created with initial payment status 'Pending'.
Log entry for the order created...
Log entry saved for Order ID 3.
Transaction successfully completed. Order placed and logged.

```

### Points to Remember:

- **TransactionManager.ImplicitDistributedTransactions:** Setting `TransactionManager.ImplicitDistributedTransactions` to true allows distributed transactions to be automatically managed if the transaction involves multiple resource managers (like different databases).
- **IsolationLevel.ReadCommitted:** Ensures that the transaction does not read data that has been modified but has not yet been committed by other transactions. This prevents scenarios like dirty reads, where uncommitted data is read.
- **Timeout:** Specifies the maximum time the transaction is allowed to run before it is automatically aborted. The default is typically sufficient, but it can be adjusted based on the application's performance requirements.
- **TransactionScopeOption.Required:** Indicates that the code block should participate in a transaction. If an existing transaction is present, it joins that transaction; otherwise, it creates a new one.
- **TransactionScopeAsyncFlowOption.Enabled:** Allows the transaction to flow correctly across asynchronous method calls, which is crucial for modern applications that rely heavily on `async/await` patterns.

## Asynchronous Transactions in Entity Framework Core

Asynchronous transactions are essential for applications that require non-blocking operations, such as web applications handling multiple concurrent requests. EF Core supports asynchronous transaction management, handling transactions without blocking the executing thread.

In a web application, handling transactions asynchronously improves scalability by freeing up threads to handle other requests while waiting for I/O operations to complete. For a better understanding, please modify the `Program` class as follows:

```
using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static async Task Main(string[] args)
        {
            try
            {
                // Initialize the ECommerceDbContext and start the
transaction
                using var context = new ECommerceDbContext();
                // Begin a transaction asynchronously
                await using var transaction = await
context.Database.BeginTransactionAsync();
                Console.WriteLine("Transaction started...");
                try
                {
                    // 1. Create a new order with order items and a pending
payment
                    var order = new Order
                    {
                        OrderDate = DateTime.UtcNow, // Set the current UTC
time for the order date
                        TotalAmount = 500.00m, // Total amount for the order
                        OrderItems = new List<OrderItem>
                        {
                            // Create a new order item for the order
```

```

        new OrderItem { ProductId = 4, Quantity = 5,
Price = 100.00m }
    },
    Payment = new Payment
    {
        PaymentDate = DateTime.UtcNow, // Set the payment
date as current UTC time
        Amount = 500.00m, // Payment amount matches the
order total
        Status = PaymentStatus.Pending // Initially set
payment status to Pending
    }
};
// Add the new order to the context
context.Orders.Add(order);
Console.WriteLine("Order and its details added to the
DbContext...");
// Save the changes asynchronously to store the order in
the database
await context.SaveChangesAsync();
Console.WriteLine("Order saved to the database with a
pending payment.");
// 2. Update the product inventory based on the order
var orderItem = order.OrderItems.First(); // Get the
first (and only) order item
var product = await
context.Products.FirstOrDefaultAsync(p => p.ProductId ==
orderItem.ProductId); // Fetch the product from the database
if (product != null)
{
    // Update the product's inventory (subtract the
ordered quantity)
    product.Quantity -= orderItem.Quantity;
    product.IsInStock = product.Quantity > 0; // Set
IsInStock based on remaining quantity
    context.Products.Update(product); // Update the
product in the context
    Console.WriteLine($"Product ID {product.ProductId}
inventory updated. Remaining stock: {product.Quantity}");
    // Save the changes asynchronously to update the
product inventory in the database
    await context.SaveChangesAsync();
    Console.WriteLine("Product inventory updated
successfully.");
}
else
{
    // If the product is not found, throw an exception
    throw new Exception("Product not found.");
}
// 3. Process the payment for the order

```

```

        order.Payment.Status = PaymentStatus.Completed; // Set
the payment status to Completed
        context.Payments.Update(order.Payment); // Update the
payment in the context
        Console.WriteLine("Payment status updated to
'Completed'...");
        // Save the changes asynchronously to update the payment
status in the database
        await context.SaveChangesAsync();
        Console.WriteLine("Payment processed and updated in the
database.");
        // Commit the transaction after all operations have
succeeded
        await transaction.CommitAsync();
        Console.WriteLine("Transaction committed successfully.");
        Console.WriteLine($"Order ID {order.OrderId} placed,
inventory updated, and payment processed successfully.");
    }
    catch (Exception ex)
    {
        // If any error occurs, roll back the transaction
        await transaction.RollbackAsync();
        Console.WriteLine($"Transaction failed. Rolling back
changes. Error: {ex.Message}");
    }
}
catch (DbUpdateException ex)
{
    // Handle database update exceptions (e.g., constraint
violations)
    Console.WriteLine($"Database Error:
{ex.InnerException?.Message ?? ex.Message}");
}
catch (Exception ex)
{
    // Handle all other exceptions
    Console.WriteLine($"Error: {ex.Message}");
}
}
}
}
}

```

## Explanation:

- **Asynchronous Methods:** To perform operations asynchronously, use `BeginTransactionAsync()`, `SaveChangesAsync()`, and `CommitAsync()`.
- **Non-Blocking:** The async approach prevents blocking threads, enhancing the application's responsiveness and scalability.
- **Error Handling:** The transaction is rolled back asynchronously if any asynchronous operation fails.

When you run the above code, you will get the following output:

```
Transaction started...
Order and its details added to the DbContext...
Order saved to the database with a pending payment.
Product ID 4 inventory updated. Remaining stock: 45
Product inventory updated successfully.
Payment status updated to 'Completed'...
Payment processed and updated in the database.
Transaction committed successfully.
Order ID 4 placed, inventory updated, and payment processed successfully.
```

## Choosing the Right Transaction Approach in EF Core

Selecting the appropriate transaction management approach in EF Core depends on your application's specific requirements and context. Consider the following factors when choosing:

### Scope of Operations:

- **Single DbContext:** Use implicit transactions via `SaveChanges()` for straightforward operations.
- **Multiple `SaveChanges()` Calls:** Use manual transactions with `BeginTransaction()` to group multiple operations.
- **Multiple DbContext Instances or Databases:** Use `TransactionScope` for distributed transactions.

### Performance and Scalability:

- **Implicit and Manual Transactions:** Generally, they have lower overhead and are suitable for most scenarios.
- **Distributed Transactions:** Higher overhead; use only when necessary as they involve coordination across multiple resources.

### Concurrency Requirements:

- **Asynchronous Transactions:** Preferable in web applications and high-concurrency environments to improve scalability and responsiveness.

For a better understanding, please have a look at the following image:

| Transaction Method        | When to Use                                                                                                      | Pros                                                  | Cons                                         |
|---------------------------|------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|----------------------------------------------|
| Implicit Transactions     | Simple operations within a single <code>DbContext</code> and single <code>SaveChanges()</code> call              | Easy to implement, minimal code                       | Limited control over transaction boundaries  |
| Manual Transactions       | Grouping multiple operations or multiple <code>SaveChanges()</code> calls within a single <code>DbContext</code> | Greater control over transaction flow                 | More verbose, requires careful management    |
| TransactionScope          | Operations spanning multiple <code>DbContext</code> instances or multiple databases/resources                    | Supports distributed transactions                     | Higher overhead, requires provider support   |
| UseTransaction()          | Sharing an existing transaction across multiple <code>DbContext</code> instances                                 | Enables transaction sharing across contexts           | Complexity in managing transaction lifetimes |
| Asynchronous Transactions | High-concurrency or I/O-bound operations, such as in web applications                                            | Non-blocking, improves scalability and responsiveness | Requires async programming patterns          |

Transactions are essential for ensuring data integrity in applications that interact with databases. EF Core offers multiple ways to handle transactions, each suitable for different scenarios. You can choose the best approach for your application's needs by understanding these methods and their appropriate use cases.

In the next article, I will discuss [Seed Data in Entity Framework Core \(EF Core\)](#) with Examples. In this article, I try to explain Transactions in Entity Framework Core (EF Core) with Examples. I hope you enjoy this Transactions in EF Core article.